

TALLER DE DISEÑO DE SOFTWARE 1

Programación Web — Paso a Paso

CLASE

03

Estructuras Condicionales

if/else, operador ternario y switch — tomando decisiones con código

Año 2026

Prof. Ing. Daniel Maldonado

Contenido de esta clase

1. ¿Qué es una condición?
2. Variables continuas vs Variables discretas
3. Operadores de comparación
4. Operadores lógicos
5. Estructura if / else if / else
6. Operador ternario
7. Estructura switch
8. ¿Cuándo usar cada estructura? — Tabla comparativa
9. Resumen final

1 ¿Qué es una condición?

Hasta ahora nuestros programas hacían siempre lo mismo — pedían datos, calculaban y mostraban el resultado. Pero en la vida real, las cosas cambian según la situación. Para eso existen las estructuras condicionales: le permiten al programa tomar decisiones.

Analogía

Pensá en un semáforo. Tiene tres posibles estados: verde, amarillo y rojo. Según el color, los autos toman una decisión diferente: avanzar, prepararse para frenar o frenar. El programa hace exactamente lo mismo: evalúa una condición y decide qué camino tomar.

¿Qué es true y false?

En programación, toda condición devuelve uno de dos valores posibles: • true → la condición es VERDADERA • false → la condición es FALSA No existe un valor intermedio. Toda comparación que hacemos en JavaScript devuelve true o false — nada más.

Por ejemplo: si una persona tiene 20 años y le preguntamos si es mayor de 18, la respuesta es true. Si le preguntamos si es mayor de 30, la respuesta es false. El programa evalúa esa condición y según el resultado ejecuta un bloque de código u otro.

2 Variables continuas vs Variables discretas

Antes de ver las estructuras condicionales, necesitamos entender un concepto fundamental que nos va a ayudar a elegir cuál usar en cada situación.

Variables continuas

Definición

Las variables continuas son aquellas que pueden asumir infinitos valores dentro de un rango de valores. No es posible enumerarlas porque entre cualquier par de valores siempre existe otro valor posible.

Ejemplos de variables continuas:

- La edad de una persona — puede ser 18, 18.5, 25, 67, 99...
- El precio de un producto — puede ser cualquier número positivo.
- La temperatura — puede ser 20°, 20.5°, 36.7°, -5°...
- El sueldo de un empleado — puede ser cualquier valor.
- La nota de un examen — puede ser 4, 6.5, 7.33, 10...

Analogía

Imaginá una regla. Entre el centímetro 1 y el centímetro 2 hay infinitos puntos posibles: 1.1, 1.5, 1.99, 1.999... No podés enumerarlos todos. Eso es una variable continua.

Variables discretas

Definición

Las variables discretas asumen valores finitos, claros y fácilmente enumerables. Se puede hacer una lista completa de todos los valores posibles sin que quede ninguno afuera.

Ejemplos de variables discretas:

- El día de la semana — lunes, martes, miércoles, jueves, viernes, sábado, domingo. Son exactamente 7 valores.
- El mes del año — enero, febrero, marzo... diciembre. Son exactamente 12 valores.
- Una opción de menú — 1, 2, 3 o 4. Valores fijos y conocidos.
- El palo de una carta — espadas, bastos, copas, oros. Son exactamente 4.
- El color de un semáforo — rojo, amarillo, verde. Son exactamente 3.

💡 Analogía

Imaginá una caja de colores con exactamente 12 colores. Podés enumerarlos todos perfectamente: rojo, azul, verde... No hay valores intermedios entre rojo y azul. Eso es una variable discreta.

La regla de oro para elegir la estructura correcta

Variable continua → usá `if / else if / else` Variable discreta → usá `switch` Esta regla te va a ahorrar muchas dudas a la hora de programar.

3 Operadores de comparación

Los operadores de comparación se usan para comparar dos valores. El resultado de cualquier comparación siempre es true o false. Son la base de todas las estructuras condicionales.

Operador	Significado	Ejemplo	Resultado
===	Igual estricto	5 === 5	true
===	Igual estricto	5 === '5'	false
!==	Distinto estricto	5 !== 3	true
!==	Distinto estricto	5 !== 5	false
>	Mayor que	10 > 5	true
<	Menor que	3 < 8	true
>=	Mayor o igual que	5 >= 5	true
<=	Menor o igual que	4 <= 3	false

¿Por qué usamos === y no ==?

JavaScript tiene dos tipos de igualdad: el == (doble igual) y el === (triple igual). La diferencia es muy importante:

Importante

El operador == compara solo el valor y puede generar resultados inesperados porque convierte los tipos automáticamente. Por ejemplo: 5 == '5' devuelve true aunque uno es número y el otro es texto. El operador === compara el valor Y el tipo de dato. Es el correcto y el que siempre debemos usar. En nuestra carrera usamos SIEMPRE === y nunca ==.

```
/* Diferencia entre == y === */  
  
console.log(5 == '5');    // true  - compara solo valor (INCORRECTO)  
console.log(5 === '5');  // false - compara valor Y tipo (CORRECTO)  
console.log(5 === 5);    // true  - mismo valor y mismo tipo  
console.log(5 !== '5');  // true  - distinto tipo
```

4 Operadores lógicos

Los operadores lógicos nos permiten combinar dos o más condiciones para formar una condición más compleja. El resultado siempre es true o false.

&& — AND (Y)

¿Qué hace?

El operador `&&` devuelve `true` SOLO cuando las DOS condiciones son verdaderas. Si alguna de las dos es `false`, el resultado es `false`.

💡 Analogía

Para entrar a un boliche necesitás DOS cosas: ser mayor de 18 Y tener DNI. Si te falta una sola de las dos, no entrás. Las dos condiciones deben cumplirse al mismo tiempo.

```
/* Ejemplos de AND (&&) */

let edad = 20;
let tieneDNI = true;

console.log(edad >= 18 && tieneDNI === true); // true - las dos son verdaderas
console.log(edad >= 18 && tieneDNI === false); // false - una es falsa
console.log(edad < 18 && tieneDNI === true); // false - una es falsa
```

|| — OR (O)

¿Qué hace?

El operador `||` devuelve `true` cuando AL MENOS UNA de las condiciones es verdadera. Solo devuelve `false` cuando las DOS son falsas.

💡 Analogía

Para pedir una pizza podés pagar con efectivo O con tarjeta. Alcanza con que tengas UNO de los dos medios de pago. Solo hay problema si no tenés ni efectivo ni tarjeta.

```
/* Ejemplos de OR (||) */

let tieneEfectivo = false;
let tieneTarjeta = true;

console.log(tieneEfectivo === true || tieneTarjeta === true); // true
- una es verdadera
console.log(tieneEfectivo === true || tieneTarjeta === false); //
false - las dos son falsas
```

! — NOT (NO)

¿Qué hace?

El operador `!` invierte el valor de una condición. Si era `true` lo convierte en `false`, y si era `false` lo convierte en `true`.

```
/* Ejemplos de NOT (!) */

let estaLloviendo = true;
```

```
console.log(estaLloviendo);    // true
console.log(!estaLloviendo);  // false - invierte el valor
console.log(!false);          // true
console.log(!true);            // false
```

Condición A	Condición B	A && B	A B
true	true	true	true
true	false	false	true
false	true	false	true
false	false	false	false

5 Estructura if / else if / else

La estructura if es la más usada en programación. Permite ejecutar un bloque de código si se cumple una condición, y otro bloque diferente si no se cumple.

¿Cuándo la usamos?

Usamos if / else cuando trabajamos con VARIABLES CONTINUAS — aquellas que pueden asumir infinitos valores dentro de un rango, como la edad, el precio, la temperatura o la nota de un examen.

Estructura básica: if / else

Analogía

Es exactamente como tomar una decisión en la vida real. Si tengo hambre, voy a comer. Si no, sigo trabajando. Hay solo dos caminos posibles.

```
/*
  Estructura básica del if / else
  Si la condición es true → ejecuta el bloque del if
  Si la condición es false → ejecuta el bloque del else
*/

if (condicion) {
  /* código que se ejecuta si la condición es true */
} else {
  /* código que se ejecuta si la condición es false */
}
```

Ejemplo completo: if / else

```
{
  /*
    Aquí le pedimos al usuario su edad
    y la guardamos en la variable edad
  */
  let edad = Number(prompt('¿Cuál es tu edad?'));

  /*
    Aquí evaluamos si la persona es mayor o menor de edad
    La edad es una variable continua – puede ser cualquier número
    Por eso usamos if / else
  */
  if (edad >= 18) {
    console.log('Sos mayor de edad. Podés ingresar.');
```

```
  } else {
    console.log('Sos menor de edad. No podés ingresar.');
```

```
  }
```

Estructura extendida: if / else if / else

Cuando tenemos más de dos posibilidades dentro de un rango continuo, usamos else if para agregar condiciones adicionales. Se evalúan en orden — en cuanto una condición es true, se ejecuta ese bloque y se ignoran los demás.

```
{
  /*
   Aquí le pedimos al usuario su nota
   y la guardamos en la variable nota
  */
  let nota = Number(prompt('¿Cuál es tu nota?'));

  /*
   Aquí evaluamos el resultado según la nota
   La nota es una variable continua — puede ser cualquier número
   entre 0 y 10
   Por eso usamos if / else if / else
  */
  if (nota >= 9) {
    console.log('Sobresaliente — Felicitaciones!');
  } else if (nota >= 7) {
    console.log('Muy bueno — Muy buen trabajo!');
  } else if (nota >= 6) {
    console.log('Bueno — Aprobaste!');
  } else if (nota >= 4) {
    console.log('Regular — Podés mejorar.');
```

⚠ Importante

Las condiciones del if / else if se evalúan de arriba hacia abajo, en orden. En cuanto una condición es true, se ejecuta ese bloque y el programa ignora todo lo que sigue. Por eso el orden importa — siempre empezá por el caso más restrictivo.

6 Operador ternario

El operador ternario es una forma compacta de escribir un if / else simple en una sola línea. Se usa cuando tenemos exactamente dos posibilidades y la lógica es sencilla.

Sintaxis del operador ternario

condicion ? valorSiEsTrue : valorSiEsFalse Se lee así: ¿Se cumple la condición? Si es true devolvé el primer valor, si es false devolvé el segundo.

Comparación: if/else vs operador ternario


```
/* Con if / else — la forma larga */
{
    let edad = Number(prompt('¿Cuál es tu edad?'));
    let mensaje;

    if (edad >= 18) {
        mensaje = 'Mayor de edad';
    } else {
        mensaje = 'Menor de edad';
    }

    console.log(mensaje);
}

/* Con operador ternario — la forma corta */
{
    let edad = Number(prompt('¿Cuál es tu edad?'));

    /*
        Aquí usamos el operador ternario:
        ¿edad >= 18? Si es true → 'Mayor de edad', si es false → 'Menor de
edad'
    */
    let mensaje = edad >= 18 ? 'Mayor de edad' : 'Menor de edad';

    console.log(mensaje);
}
```

Importante

El operador ternario solo se usa cuando la lógica es simple y hay exactamente DOS opciones. Si necesitás más de dos opciones o la lógica es compleja, usá if / else if / else — el código tiene que ser fácil de leer.

Otro ejemplo con operador ternario

```
{
    let numero = Number(prompt('Ingresá un número:'));

    /*
        Aquí usamos el operador ternario para determinar
        si el número es par o impar
        Un número es par si el resto de dividirlo por 2 es 0
    */
    let resultado = numero % 2 === 0 ? 'El número es par' : 'El número
es impar';

    console.log(resultado);
}
```

7 Estructura switch

La estructura switch evalúa una variable y compara su valor contra una lista de casos posibles. Cuando encuentra una coincidencia, ejecuta el bloque de código de ese caso.

¿Cuándo la usamos?

Usamos switch cuando trabajamos con VARIABLES DISCRETAS — aquellas que asumen valores finitos, claros y fácilmente enumerables, como el día de la semana, el mes, una opción de menú, o el palo de una carta.

Estructura del switch

```
/*
  Estructura básica del switch
  switch evalúa la variable y busca el case que coincida
  break corta la ejecución — sin él se ejecutarían todos los casos
  siguientes
  default es el caso por defecto — se ejecuta si ningún case coincide
*/

switch (variable) {
  case valor1:
    /* código para el caso 1 */
    break;
  case valor2:
    /* código para el caso 2 */
    break;
  default:
    /* código si ningún caso coincide */
}
}
```

Ejemplo completo: días de la semana

```
{
  /*
    Aquí le pedimos al usuario un número del 1 al 7
    que representa el día de la semana
    El día de la semana es una variable discreta — exactamente 7
    valores posibles
    Por eso usamos switch
  */
  let dia = Number(prompt('Ingresá un número del 1 al 7 (1=Lunes,
7=Domingo):'));

  switch (dia) {
    case 1:
      console.log('Lunes — Inicio de la semana laboral.');
```

```
        console.log('Martes');
        break;
    case 3:
        console.log('Miércoles — Mitad de semana.');
```

```
        break;
    case 4:
        console.log('Jueves');
```

```
        break;
    case 5:
        console.log('Viernes — ¡Casi fin de semana!');
```

```
        break;
    case 6:
        console.log('Sábado — Fin de semana.');
```

```
        break;
    case 7:
        console.log('Domingo — Último día del fin de semana.');
```

```
        break;
    default:
        console.log('Número inválido. Ingresá un número del 1 al
7.');
```

```
    }
}
```

Ejemplo: menú de opciones

```
{
    /*
    Aquí mostramos un menú con opciones al usuario
    Las opciones son una variable discreta — valores fijos y
    enumerables
    */
    let opcion = Number(prompt('Seleccioná una opción:\n1. Ver saldo\n2.
Depositar\n3. Extraer\n4. Salir'));

    switch (opcion) {
        case 1:
            console.log('Tu saldo actual es: $15000');
```

```
            break;
        case 2:
            console.log('Ingresá el monto a depositar.');
```

```
            break;
        case 3:
            console.log('Ingresá el monto a extraer.');
```

```
            break;
        case 4:
            console.log('Hasta luego.');
```

```
            break;
        default:
            console.log('Opción inválida. Seleccioná una opción del 1 al
4.');
```

```
    }
}
```

**Importante**

El break al final de cada case es obligatorio. Sin el break, el programa continúa ejecutando los casos siguientes aunque no coincidan — esto se llama 'fall-through' y casi siempre genera errores. Nunca olvides el break.

8 ¿Cuándo usar cada estructura?

Elegir la estructura correcta hace que el código sea más claro, más fácil de leer y de mantener. Esta tabla resume la regla de oro:

Estructura	Tipo de variable	¿Cuándo la uso?	Ejemplo de uso
if / else if / else	Variable continua	Cuando los valores posibles son infinitos dentro de un rango	Edad, precio, temperatura, nota
Operador ternario	Variable continua	Cuando hay exactamente DOS opciones y la lógica es simple	¿Es par o impar? ¿Es mayor o menor?
switch	Variable discreta	Cuando los valores posibles son finitos y enumerables	Día de la semana, mes, opción de menú

Resumen rápido

- ¿Los valores posibles son infinitos o están dentro de un rango? → usá if / else if / else
- ¿Hay exactamente dos posibilidades y la lógica es simple? → podés usar el operador ternario
- ¿Podés hacer una lista completa de todos los valores posibles? → usá switch

Analogía

Si tenés que decidir qué hacer según la temperatura, usás if/else — la temperatura puede ser cualquier número. Si tenés que decidir qué hacer según el mes del año, usás switch — los meses son exactamente 12, finitos y enumerables.



Resumen — Todo lo que aprendiste hoy

Concepto	¿Qué es?
Condición	Una pregunta cuya respuesta es siempre true (verdadero) o false (falso).
true / false	Los únicos dos valores posibles al evaluar una condición.
Variable continua	Puede asumir infinitos valores dentro de un rango. Ej: edad, precio.
Variable discreta	Asume valores finitos, claros y enumerables. Ej: día de la semana.
===	Igual estricto — compara valor Y tipo de dato. Siempre usamos este.
!==	Distinto estricto — devuelve true si el valor o el tipo son diferentes.
> / < / >= / <=	Operadores de comparación de mayor, menor, mayor o igual, menor o igual.
&& (AND)	Devuelve true solo si las DOS condiciones son verdaderas.
(OR)	Devuelve true si AL MENOS UNA condición es verdadera.
! (NOT)	Invierte el valor — true se convierte en false y viceversa.
if / else if / else	Estructura para variables continuas. Evalúa condiciones en orden.
Operador ternario	Forma corta de if/else para dos opciones simples. condicion ? a : b
switch	Estructura para variables discretas. Compara contra valores fijos.
break	Termina la ejecución de un case en el switch. Siempre obligatorio.
default	Caso por defecto del switch. Se ejecuta si ningún case coincide.

¿Qué viene en la Clase 04?

Estructuras repetitivas — los bucles. Vamos a ver cómo hacer que el programa repita una acción muchas veces sin tener que escribir el mismo código una y otra vez. Veremos for, while y do/while.